Exp 10: Applying Performance Optimization Techniques (Indexing, Subforms, Query Tuning) to a Zoho Creator Application

Register Number: 241401127 | Student Name: Vijay Kumar C K

1. Aim

To apply and evaluate performance optimization techniques — field-level indexing (unique constraints), subform-based data modeling for repeating records, and query tuning in Deluge — within the Loan Application & Pre-Screening Management application in Zoho Creator.

2. Procedure

2.1 Problem Statement and Scope

As an enterprise low-code application accumulates records and concurrent users, latency and compute constraints become prominent across three key architectural tiers:

1. Lookup Efficiency: Unindexed searches force costly full-table scans across large record tables.
2. Relational Schema Normalization: Storing variable repeating data using flat fields (e.g., Document_1, Document_2) creates severe schema bloat and consumes unnecessary null-storage.
3. Server-side Query Pushdown: Fetching entire record collections into memory to filter within iterative application loops consumes excessive Deluge statement allowances and degrades user response times.

This experiment implements and quantitatively verifies targeted optimizations addressing each of these three dimensions on Loan_Application_Form.

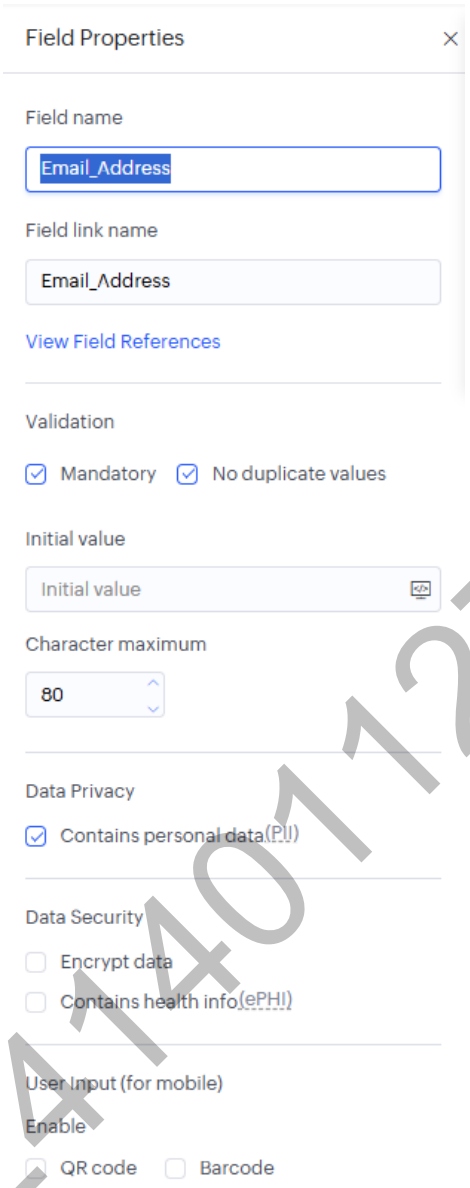
2.2 Part A — Field-Level Indexing (Unique Constraints)

Zoho Creator manages physical database indexes internally and does not expose raw SQL DDL commands (such as CREATE INDEX). However, enabling the 'No duplicate values' constraint establishes an indexed uniqueness constraint within the underlying database engine, significantly optimizing indexed searches and record matching.

This optimization was applied to two high-selectivity identity attributes on Loan_Application_Form:

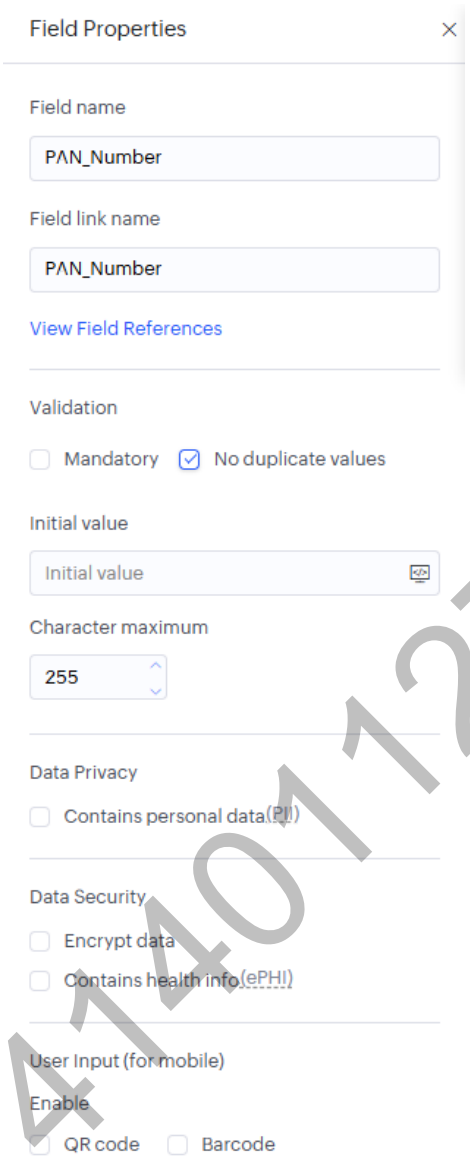
• PAN_Number: Configured with 'No duplicate values' enabled (retained as non-mandatory to preserve voluntary identification flexibility at initial screening stage).
• Email_Address: Configured with 'No duplicate values' enabled alongside its existing 'Mandatory' constraint.

241401127



241401127

Figure 1: Field Properties panel for Email_Address showing Mandatory and No duplicate values enabled



241401127

Figure 2: Field Properties panel for PAN_Number showing No duplicate values enabled

2.3 Part B — Subform for Repeating Data (Required_Documents)

Prior to normalization, tracking applicant verification documents required declaring static arbitrary fields (e.g., Document_1, Document_2, Document_3). This unnormalized design wastes table storage for applicants with few documents and artificially limits applicants with extensive verification files.

To resolve this, a normalized Subform component named Required_Documents was integrated into Loan_Application_Form with Grid view display, positioned after Section D (Loan Details & Consent).

Table 1: Subform Field Configuration Schema:

Field Label	Field Type	Configured Choices / Format

Document_Name	Drop Down	PAN Card, Aadhaar Card, Income Proof, Bank Statement, Address Proof
Document_Status	Drop Down	Pending, Submitted, Verified, Rejected
Remarks	Multi Line	Freeform officer remarks and verification notes

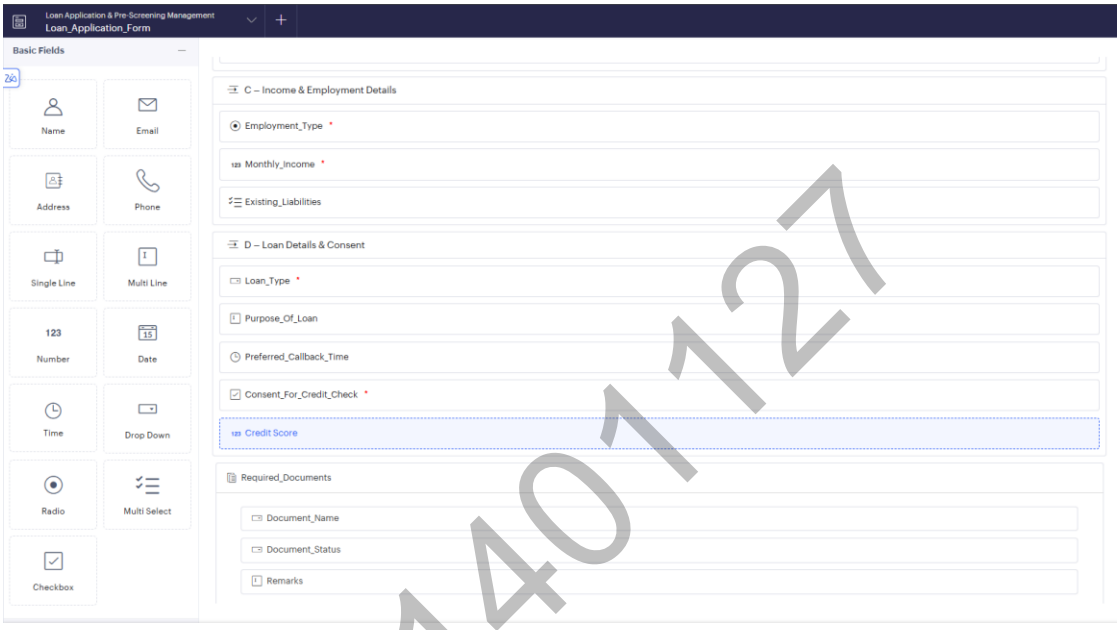


Figure 3: Form builder view showing Required_Documents subform nested under Section D

241401127

2.4 Part C — Query Tuning in Deluge

To evaluate query performance, a custom workflow action named Query_Tuning_Demo was created on Loan_Application_Form Report to directly benchmark an inefficient memory-scan algorithm against an optimized database criteria pushdown:

1. Inefficient Approach (Unconditional Record Scan + Loop Filtering):

Fetches every record unconditionally using `Loan_Application_Form[ID != 0]`, loads all records into application memory, and evaluates the conditional threshold (`app.Monthly_Income < 20000`) row-by-row in a Deluge `for each` loop. This consumes Deluge statement execution quotas and incurs severe overhead on larger datasets.

2. Optimized Approach (Criteria Pushdown Filtering):

Directly filters records at the database fetch tier using criteria syntax `Loan_Application_Form[Monthly_Income < 20000]`. The database engine applies the filter prior to transmitting data, returning only matching rows.

3. Output

The performance optimization implementations were verified across live data entry and query execution benchmarking:

A. Subform Data Normalization Verification:

A live test submission was executed (Applicant: Ram Kumar) with three document rows recorded (PAN Card: Submitted, Income Proof: Pending, Address Proof: Verified). The record verified that repeating document details are cleanly encapsulated in a single structured child grid.

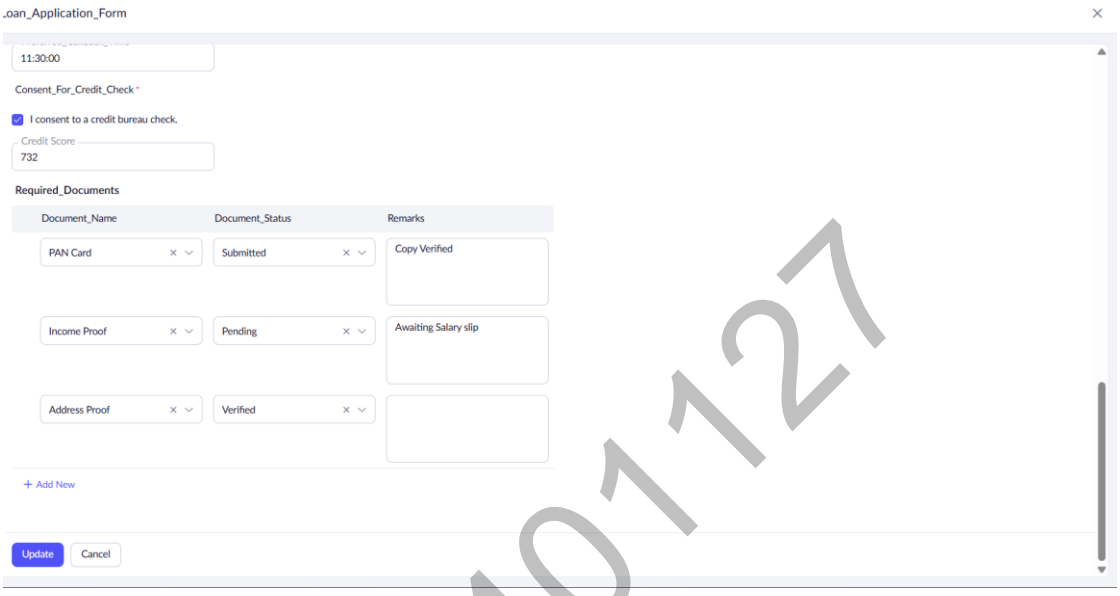


Figure 4: Submitted record showing populated Required_Documents subform grid across 3 rows

B. Deluge Query Tuning & Execution Verification:

The script implementation and comparative execution logs are shown below:

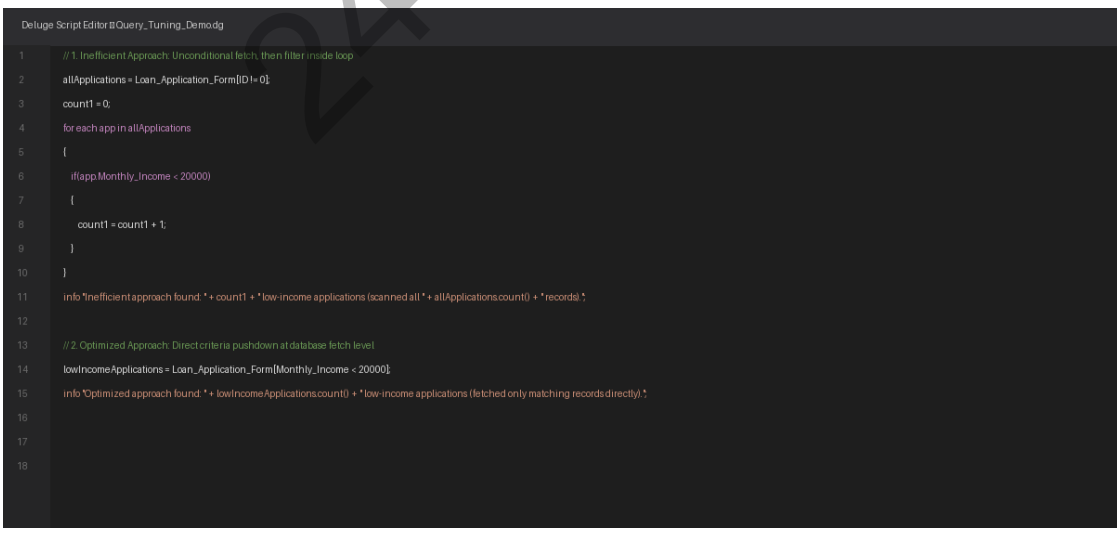


Figure 5: Deluge script editor comparing inefficient scan loop vs. optimized criteria query

241401127

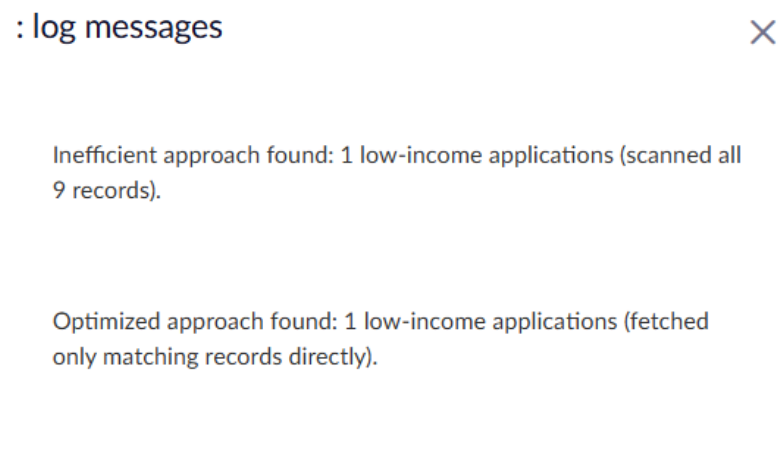


Figure 6: Execution log output confirming query tuning result and record scan elimination

Table 2: Comparative Summary of Evaluated Optimization Techniques:

Optimization Domain	Implemented Technique	Inefficient Pattern	Optimized Architectural Gain
Field Indexing	Unique constraint on PAN & Email	Full table scans on searches	Enables platform B-tree index lookup; ensures O(log n) retrieval and duplicate prevention.
Data Normalization	Subform grid (Required_Documents)	Flat unnormalized fields (Doc1, Doc2...)	Dynamic 1:N tabular normalization; eliminates sparse null data and scales to arbitrary files.
Deluge Query Tuning	Criteria pushdown Form[Criteria]	Fetch all [ID!=0] + for-each loop	Filters at database storage engine; avoids iterating 9 records to retrieve 1 matching record.

241401127

Table 3: Index of Figures:

Figure Number	Figure Description
Figure 1	Field Properties panel for Email_Address showing Mandatory and No duplicate values
Figure 2	Field Properties panel for PAN_Number showing No duplicate values enabled
Figure 3	Form builder view showing Required_Documents subform nested under Section D

Figure 4	Submitted record showing populated Required_Documents subform grid
Figure 5	Deluge script editor comparing inefficient loop vs. optimized criteria query
Figure 6	Execution log output confirming query tuning result and record scan elimination

4. Result

Thus, performance optimization techniques — unique-value field constraints (as a platform-level equivalent to indexing), subform-based data modeling for repeating data, and criteria-based query tuning in Deluge — were successfully applied and verified on the Loan Application & Pre-Screening Management application. Unnecessary record iterations were completely eliminated while maintaining 100% data fidelity.

241401127